YOUR PROJECT / YOUR LEARNING PATH

# Angular project complete tutor guide

Understand the concepts.
Recognize the patterns.
Then walk through the code.

## Order Management Practice Lab

Based on your simplified Angular 21 implementation, with TypeScript foundations, every application feature, code-reading checkpoints and practice exercises.

| LEARNING LEVEL | PROJECT SNAPSHOT |
| --- | --- |
| Beginner to confident code reader | 9 November 2026 |
| START WITH | STACK |
| Templates, signals and the catalog | Angular 21 / ASP.NET Core / SQL Server |

CONTENTS / LEARNING MAP

# Contents

Read in sequence, or click a chapter title to jump to it. PDF bookmarks provide the same navigation.

START HERE

01 How to use this tutor guide 4

02 Everything the application can do 5

03 Roles, permissions and routes 6

04 Architecture: where each responsibility belongs 7

05 The frontend file map 8

FOUNDATIONS

06 Startup: index.html, main.ts and app.config.ts 9

07 TypeScript basics you need first 10

08 Expressions, objects and arrays 11

09 Components, standalone imports and injection 12

10 Signals: storing visible state 13

11 computed, effect and change detection 14

12 Lifecycle and subscription cleanup 15

13 Template binding: read, set and react 16

14 Template control flow and stable rendering 17

15 Pipes and formatting 18

16 Reactive forms: individual controls 19

17 FormGroup, validation and submission 20

18 HTTP services and typed API contracts 21

19 Observable, subscribe and Subscription 22

20 RxJS operators: the small set to learn first 23

21 Advanced RxJS retained for authentication 24

22 Routing and lazy-loaded pages 25

23 Guards and safe navigation 26

24 Shared components, inputs, outputs and directives 27

FEATURE WALKTHROUGHS

25 The root component and shared layout 28

26 Products.ts: the first complete class to learn 29

CONTENTS / LEARNING MAP

# Contents continued

Read in sequence, or click a chapter title to jump to it. PDF bookmarks provide the same navigation.

FEATURE WALKTHROUGHS

27 Catalog HTML and one search traced end to end 30
28 Account component: one form, several operations 31
29 Account operations and their outcomes 32
30 Auth service: user state, login and restoration 33
31 HTTP interceptor, loading count and error propagation 34
32 Cookies, CSRF and shared refresh 35
33 Orders list: filters and pagination 36
34 CreateOrder: catalog selection and local cart 37
35 OrderDetail: edits, cancellation and status 38
36 Product administration: CRUD with a soft delete 39
37 User administration: roles and locks 40

INTEGRATION AND PRACTICE

38 Backend concepts Angular needs to understand 41
39 Business rules: totals, stock and order status 42
40 Errors, empty results and safe retries 43
41 Styling, responsive layout and accessibility 44
42 Testing: what the project verifies 45
43 Debugging a feature without getting lost 46
44 Running, building and configuration 47
45 Hosting and delivery: enough context for Angular 48
46 A guided learning sequence 49
47 Practice exercises 50
48 Self-check answers and common misconceptions 51

REFERENCE

49 Glossary for your walkthrough 52
50 Coverage checklist: concepts and where to find them 53
51 Source index and further reading 54

01 / START HERE

# How to use this tutor guide

CODE MAP: Current frontend source and frontend/package.json

This book prepares you to read your Order Management project with confidence. It explains the Angular and TypeScript constructs that actually appear in the code, then connects each feature to its component, template and API. It follows the simplified implementation created in this conversation.

## Study in three passes

- First pass: read the foundations. Learn what a component, template, service, signal, form control and Observable do. You do not need to memorize authentication internals yet.
- Second pass: follow products from the HTML input to the API and back. Then read account, orders and administration using the same pattern.
- Third pass: study refresh coordination, tests, conflicts and deployment context. These explain why some infrastructure remains more advanced.

## What the guide covers

All application screens, the Angular concepts used by them, supporting TypeScript syntax, API contracts, browser security, testing and debugging. Backend rules and hosting are explained where they affect the Angular app. This is an Angular project guide, not a complete C#, SQL Server or Azure reference.

## What counts as an example

Code blocks labelled "Project excerpt" are copied from the current source, sometimes with surrounding lines omitted. Blocks labelled "Teaching example" demonstrate the same idea in fewer lines. They are explanations, not additional changes to your app.

Your project pins Angular 21.0.0, TypeScript 5.9.3 and RxJS 7.8.2. Modern tutorials may use other APIs. Learn this project's reactive forms and explicit loading methods before trying alternatives. Source reviewed on 9 November 2026; later edits can change the details.

> Tutor habit: before opening a method, predict its inputs, the state it changes, the request it makes and the visible result. Then verify your prediction in the debugger.

02 / START HERE

# Everything the application can do

CODE MAP: features/products, features/account, features/orders, features/admin

The application is an order-management practice lab. Angular provides the browser interface; ASP.NET Core provides HTTP endpoints; SQL Server stores business and account data.

| Feature | What you can do | Main component |
|---|---|---|
| Public catalog | Search name/SKU, sort, browse pages, inspect stock and price | Products |
| Account | Register, confirm email, sign in/out, recover and change passwords | Account, App |
| Order list | Search, filter status, sort, browse and open details | Orders |
| New order | Search products, build a cart, change quantities, submit a Draft | CreateOrder |
| Order details | Edit eligible Draft items, cancel, advance permitted status | OrderDetail |
| Product admin | Create/edit products and deactivate them | ProductAdmin |
| User admin | Search users, assign roles and lock/unlock accounts | Users |
| Shared feedback | Display loading, errors, success and pagination | App, Feedback, Pager |

## Features that work behind the screen

- Session restoration when the browser reloads.
- A shared authentication refresh when eligible API requests return 401.
- CSRF request headers and cookie-based authentication.
- Route guards and permission-based link visibility.
- Cancellation of older read requests and delayed search while typing.
- Server validation, inventory transactions and concurrency conflicts.

## Your first demonstration

Open Catalog, search for an existing product, sort by price and switch pages. Sign in, create a small order and open its details. If you have a staff account, inspect product management. With an Admin account, inspect Users. Avoid changing your own account while learning role management.

> Checkpoint: can you distinguish a visible feature, such as search, from an infrastructure feature, such as request cancellation?

03 / START HERE

# Roles, permissions and routes

CODE MAP: app.config.ts; core/guards.ts; shared/ui.ts; API Security/AuthSupport.cs

A role is an account category. A permission names an allowed operation. The backend turns Admin and Manager roles into ManageProducts, ViewAllOrders and ManageOrderStatus permission claims. Customer has no staff permissions.

| Capability | Customer | Manager | Admin |
|---|---|---|---|
| Browse and create own orders | Yes | Yes | Yes |
| Edit own Draft items | Yes | Yes | Yes |
| View all orders | No | Yes | Yes |
| Manage products and advance status | No | Yes | Yes |
| Edit another owner's Draft items | No | No | Yes |
| Manage users and roles | No | No | Yes |

## Route map

- / redirects to /products. Catalog is public.
- /account/login, /register, /forgot-password, /reset-password, /confirm-email and /resend-confirmation are under /account.
- /account/change-password requires sign-in.
- /orders, /orders/new and /orders/:id require sign-in.
- /admin/products requires sign-in and ManageProducts.
- /admin/users requires sign-in and Admin status.
- /access-denied explains denied navigation; ** matches unknown URLs.

## Three separate checks

The permission directive decides whether a link is rendered. A guard decides whether Angular navigation is allowed. The API decides whether the request is authorized. Only the API can protect stored data against a caller who bypasses the interface.

Managers may read all orders without being allowed to edit another customer's Draft items. The order's owner, status and server rules matter in addition to general permissions. Cancellation has its own server authorization and transition checks.

> Checkpoint: hiding the Users link does not secure /api/admin/users. Explain which layer enforces that endpoint.

04 / START HERE

# Architecture: where each responsibility belongs

CODE MAP: frontend/src/app; src/OrderManagement.Api; Application; Infrastructure; Domain

Think of each feature as a small conversation between the screen and the server.

```
User action in HTML
        |
Component method and form state
        |
Angular HTTP service -> API interceptor
        |
ASP.NET controller -> application service
        |
Repository / EF Core -> SQL Server
        |
JSON response -> signal update -> rendered HTML
```

## Browser responsibilities

The template displays data and binds actions. The component coordinates UI state and requests. An API service gives requests meaningful names such as list(), create() or cancel(). Shared services keep authentication and HTTP activity available across screens.

## Server responsibilities

Controllers handle HTTP and authorization. Application services enforce business operations. Infrastructure implements persistence and external services. Domain classes describe products, orders and items. A client-supplied total or permission is never trusted as authority.

## One concrete example

When you change the catalog sort, the FormControl emits a new value. Products.applyFilters() selects page 1. loadProducts() calls ProductApi.list(). The server returns a Page<Product>. The component calls data.set(result). The @for block renders the new items.

The response travels through the interceptor before reaching the component subscriber. The interceptor can refresh authentication or report a failure, while the screen still owns its local loading/error state.

> Do not put SQL queries in a component. Angular runs in the browser and talks to HTTP endpoints; it does not connect directly to SQL Server.

05 / START HERE

# The frontend file map

CODE MAP: frontend/src; frontend/angular.json; frontend/tsconfig.json

| File or folder | Read it to understand |
| --- | --- |
| src/index.html | Browser document and app-root host |
| src/main.ts | Angular startup |
| app/app.config.ts | HTTP providers, session initializer and routes |
| app/app.ts + app.component.html | Root layout, navigation and logout |
| app/core/auth.ts | User state and account HTTP flows |
| app/core/http.ts | Request preparation, loading count and refresh retry |
| app/core/guards.ts | Navigation access checks |
| app/shared/ui.ts | Feedback, Pager and Permission directive |
| app/features/products | Public catalog component, HTML and API types |
| app/features/account | Reused account form and its template |
| app/features/orders | List, create, detail, templates and API service |
| app/features/admin | Product and user management |
| src/styles.css | Global layout and responsive styling |
| *.spec.ts | Frontend tests |

## Names are conventions

Your catalog file is products.ts, not product.ts. Your root class is App in app.ts, not AppComponent in app.component.ts. Angular reads imports and decorator metadata; it does not require a particular component filename suffix.

## Configuration around the source

package.json lists dependencies and npm scripts. angular.json describes builds, serving and tests. tsconfig.json enables TypeScript and template strictness. proxy.conf.json sends development /api requests to the local API. package-lock.json records resolved npm dependency versions for reproducible installation.

> First reading order: index.html -> main.ts -> app.config.ts -> app.ts -> app.component.html -> products.ts -> products.component.html -> product-api.ts.

06 / FOUNDATIONS

# Startup: index.html, main.ts and app.config.ts

CODE MAP: src/index.html; src/main.ts; app/app.config.ts

## The browser document

index.html sets the title, UTF-8 encoding, viewport and base URL. Its body contains <app-root>. That element is the host Angular uses for the root component. <base href="/"> supports navigation relative to the app root.

Project excerpt from main.ts:

```
import { bootstrapApplication } from "@angular/platform-browser";
import { appConfig } from "./app/app.config";
import { App } from "./app/app";
bootstrapApplication(App, appConfig).catch(console.error);
```

bootstrapApplication starts a standalone application using App and appConfig. It returns a Promise; catch(console.error) reports startup failure. The build system adds the JavaScript bundles to the generated HTML; your source index.html does not manually list main.ts as a browser script.

## Application providers

provideHttpClient enables HTTP requests and installs interceptors/XSRF configuration. provideRouter supplies URL matching and navigation. provideAppInitializer(restoreSession) waits for session restoration before application startup completes.

restoreSession injects Auth and uses firstValueFrom(auth.restore()). That converts the first Observable emission into a Promise. An unauthenticated/offline startup can resolve to null, allowing the public catalog to start instead of failing the whole application.

## Root connection

App declares selector: "app-root" and templateUrl: "./app.component.html". The root HTML contains an active router-outlet in the reviewed source. Routed pages appear there; the surrounding header/footer remain.

> Checkpoint: main.ts starts Angular; app.config.ts supplies application services and routes; index.html supplies the browser document. app.config.html is not a file in this project.

07 / FOUNDATIONS

# TypeScript basics you need first

CODE MAP: All frontend .ts files; product-api.ts; order-api.ts

TypeScript adds compile-time types to JavaScript. The build produces JavaScript for the browser. Types help you catch mistakes while editing, but they do not validate JSON returned by a server at runtime.

| Construct | Meaning in this project |
|---|---|
| import / export | Share classes, functions and types between files |
| class Products | Groups state and methods for the catalog |
| interface Product | Describes the expected product object's shape |
| string, number, boolean | Text, numeric values and true/false |
| Product[] | An array of products |
| User \| null | A user object or no user |
| Page<Product> | A generic page whose items are products |
| private | Implementation member used inside the class |
| readonly | Prevents reassignment of that property |
| implements OnInit | Checks that the class supplies the required method |

Teaching example:

```
interface Product {
  id: number;
  name: string;
  price: number;
}

function lineTotal(price: number, quantity: number) {
  return price * quantity;
}
```

The type after a colon describes a value or parameter. TypeScript often infers return types, so a missing explicit return type does not mean "untyped".

## readonly does not freeze everything

readonly error = signal("") prevents assigning a different signal to error. Calling error.set("Failed") is valid. readonly auth also does not freeze all state inside the injected Auth service.

## DTOs and generics

DTO means data transfer object: a shape used across the HTTP boundary. Page<T> allows the same pagination shape to contain Product, Order or AdminUser items. HttpClient.get<Page<Product>>() tells TypeScript what you expect; it does not prove the response follows that contract.

08 / FOUNDATIONS

# Expressions, objects and arrays

CODE MAP: account.ts; create.ts; core/auth.ts; shared/ui.ts

| Expression | Read it aloud |
| --- | --- |
| auth.user()?.email | Read email if a user exists |
| title ?? "Request failed" | Use the fallback only for null/undefined |
| !busy() | The opposite of busy |
| allowed && signedIn | Both conditions must be true |
| isAdmin \|\| isOwner | At least one condition is true |
| condition ? a : b | Choose a if true, otherwise b |
| { ...line, quantity } | Copy line and replace quantity |
| [ ...lines, newLine ] | Create an array with an extra element |
| `${id}` in a template string | Insert the value into text |

Project-style examples:

```
const { email, password } = this.form.getRawValue();
const existing = this.cart().find(line => line.product.id === id);
const remaining = this.cart().filter(line => line.product.id !== id);
```

Destructuring extracts named properties. find returns the first matching item or undefined. filter returns a new array of matching items. map transforms each array item; it is used to build request objects and replace changed cart lines. includes checks whether an array contains a value.

## Arrow functions

line => line.product.id === id is a short function. An expression body returns automatically. With braces, use return when you need to produce a value. Arrow callbacks also retain the surrounding this, which is helpful in subscriptions.

## Optional members and assertions

private productRequest?: Subscription can be undefined until a request starts. productRequest?.unsubscribe() safely does nothing before that. An as type assertion changes TypeScript's interpretation, not the underlying data. Avoid using assertions to hide a missing runtime check.

void router.navigate(...) intentionally discards a returned Promise value. It does not wait for navigation. Number(...) and the unary + used in templates convert text from HTML inputs to numbers.

09 / FOUNDATIONS

# Components, standalone imports and injection

CODE MAP: app.ts; products.ts; product-api.ts; shared/ui.ts

Project excerpt:

```
@Component({
  standalone: true,
  imports: [ReactiveFormsModule, CurrencyPipe, RouterLink],
  templateUrl: "./products.component.html",
})
export class Products implements OnInit, OnDestroy {
```

The decorator supplies Angular metadata. The class holds behavior and state; the template defines the view. The catalog is loaded by a route, so you do not manually insert it into another template using a selector. App, Feedback and Pager have explicit selectors because their host elements are used in HTML.

## Two different kinds of imports

A TypeScript import makes a symbol available in a .ts file. The component's imports array makes standalone directives, pipes and components available to its HTML. Importing CurrencyPipe at the top without including it in component metadata would not make the currency pipe available to that template.

## Dependency injection

private readonly api = inject(ProductApi) asks Angular for a configured service. You do not call new ProductApi(). Angular supplies that service's HttpClient dependency as well. Injection in field initializers works while Angular is creating the component or service; an arbitrary later function call is not automatically an injection context.

@Injectable({ providedIn: "root" }) registers services such as Auth and ProductApi at application scope, normally giving all screens the same instance in that root injector. Component fields such as cart or error belong to the individual component instance.

## Provider versus template import

provideHttpClient configures a service at application level. ReactiveFormsModule supplies template form directives. These solve different needs even though both are part of Angular setup.

> Checkpoint: why is the shared Auth service useful when both the header and an account screen need the same user state?

10 / FOUNDATIONS

# Signals: storing visible state

CODE MAP: products.ts; app.ts; core/auth.ts; create.ts

A signal holds a current value. Angular tracks reads of signals in templates and updates the relevant view when the value changes. This project uses signals for products, errors, loading, user details, pages and cart lines.

Project-style example:

```
readonly loading = signal(true);
readonly error = signal("");

this.loading.set(false);
this.error.set("Could not load products.");
```

In HTML, loading() and error() read the values. The parentheses matter: error is the signal object; error() is its current string.

## set versus update

set replaces the value directly. update calculates the next value from the previous value. Products.move() uses page.set(nextPage); cart additions use cart.update(lines => [...lines, newLine]).

## Arrays and object references

To update a cart line, create a new array and a new object for the changed line. Do not silently mutate this.cart()[0].quantity and expect the signal to notify consumers. A new reference makes the state change observable to the signal's equality check.

## Signals versus plain fields

statuses is a plain array because it is a fixed list of labels. loading is a signal because asynchronous callbacks change it and the template needs to react. A form control has its own value/state API; search.value is not a signal read.

## State ownership

Products.data belongs to the catalog screen. Auth.user belongs to a root service and is shared across screens. After a full browser reload, memory is recreated; the initializer restores the user using server cookies, rather than assuming a signal survives reloads.

> Practice: change a signal's initial value in an isolated exercise, then predict which HTML branch will render. Restore the original value afterward.

11 / FOUNDATIONS

# computed, effect and change detection

CODE MAP: products.ts; create.ts; detail.ts; shared/ui.ts; angular.json

computed derives a value from other signals. It is read-only from the consumer's point of view; you update the source state rather than setting the computed result.

Project excerpt:

```
readonly pages = computed(() =>
  Math.max(1, Math.ceil(this.data().totalCount / this.data().pageSize)),
);
```

For 25 products and page size 12, Math.ceil produces 3 pages. Math.max ensures at least 1. CreateOrder.total computes a preview from cart quantities and prices. OrderDetail.canEdit and nextStatus derive available UI actions from order and authentication state.

## effect performs a side effect

Permission uses effect because it must create or remove an embedded view. The effect reads the permission input and authentication signals. When their dependencies change, Angular reruns it. view.clear() prevents duplicate inserted content before the permitted view is created again.

Use computed for derived data; use effect for work outside a simple calculation, such as synchronizing a non-reactive view API. Do not use an effect merely to copy one signal's value into another when a computed value would express the relationship.

## How the view updates

Angular evaluates template bindings and reconciles the rendered DOM with application state. Signals provide notifications for state changes. The reviewed setup has no zone.js dependency or configured zone.js polyfill; Angular 21 uses its default zoneless setup here. The components do not explicitly choose OnPush.

That makes signal updates important in asynchronous callbacks. Calling data.set(result) tells Angular about newly received data. A plain field changed in arbitrary asynchronous code is not a reliable replacement for a signal in this setup.

> Checkpoint: pages is derived state; the pending request count is stored state. Explain why they are represented differently.

12 / FOUNDATIONS

# Lifecycle and subscription cleanup

CODE MAP: products.ts; orders.ts; account.ts; admin/products.ts

A component is created, initialized, displayed and eventually destroyed. Field initializers prepare its dependencies and state. ngOnInit runs once per instance after Angular initializes its inputs. ngOnDestroy runs before that instance is destroyed.

The simplified catalog uses an explicit pattern:

```
ngOnInit() {
  this.loadProducts();
  // Register search and sort subscriptions.
}

ngOnDestroy() {
  this.subscriptions.unsubscribe();
  this.productRequest?.unsubscribe();
}
```

Subscription.add groups form listeners so they can all be unsubscribed together. The active HTTP request is stored separately because every new load cancels the previous read request before replacing it.

## The other cleanup style in this project

Many components keep DestroyRef and apply takeUntilDestroyed(this.destroy). That operator unsubscribes automatically at destruction. It is the same cleanup goal expressed through an Angular helper instead of a handwritten ngOnDestroy.

Passing DestroyRef explicitly allows the operator to be used inside methods called later, such as load() or save(). Those methods do not need to run in a fresh injection context.

## Why cancel requests?

Suppose an old search is slow and a newer search finishes first. Without cancellation, the old result could overwrite the new result. Unsubscribing from an HttpClient request stops your subscription and normally aborts the browser request. It does not guarantee the server stopped all work, and it cannot undo a completed server write.

Do not confuse HTTP completion with form listeners. HTTP requests usually finish; valueChanges can keep emitting for the lifetime of the form. Both need appropriate cleanup if navigation happens while they are active.

13 / FOUNDATIONS

# Template binding: read, set and react

CODE MAP: app.component.html; products.component.html; users.component.html

Angular templates extend HTML with expressions and bindings. The expressions read public members of the component class.

| Syntax | Direction | Example |
|---|---|---|
| {{ expression }} | Component -> displayed text | {{ product.name }} |
| [property]="expression" | Component -> element/component property | [disabled]="busy()" |
| (event)="method()" | User/child event -> component | (click)="logout()" |
| [class.name]="condition" | State -> CSS class | [class.sold-out]="!product.stockQuantity" |
| #reference | Name an element in the template | #qty on an input |
| $event | Read the emitted event value | load($event) |

## Attribute text versus property expression

class="primary" is static text. [disabled]="busy()" evaluates a boolean. Writing disabled="false" is not equivalent: the presence of a boolean HTML attribute can still disable an element.

## Template references

In order creation, #qty names an HTMLInputElement. qty.value is text, so +qty.value converts it to a number before quantity() validates it. The Admin users template similarly reads role.value from #role on a select.

## Parentheses do not always mean a signal

loading() reads a signal, but pages() in some components is an ordinary method. Both are callable from HTML. Inspect the TypeScript declaration to distinguish signal(...), computed(...) and a normal method.

## Forms are their own connection

[formControl]="search" connects a FormControl to an input. The application does not use [(ngModel)] for these screens. Learn its existing reactive form bindings before adding template-driven forms.

> Teaching exercise: translate [disabled]="busy() || !cart().length" into words: disable when a request is busy or the cart is empty.

14 / FOUNDATIONS

# Template control flow and stable rendering

CODE MAP: products.component.html; order-detail.component.html; app.component.html

## Conditional blocks

Project-style excerpt:

```
@if (loading()) {
  <p>Loading products...</p>
} @else if (error()) {
  <button (click)="loadProducts()">Try again</button>
} @else {
  <!-- Product results -->
}
```

Only the matching branch is rendered. The catalog prioritizes loading over error and displays products only after a successful request. The root header uses @if / @else to switch between signed-in links and Sign in/Register.

## Repeating items

```
@for (product of data().items; track product.id) {
  <h3>{{ product.name }}</h3>
} @empty {
  <p>No matching products.</p>
}
```

@for repeats a block for each item. track product.id gives Angular a stable identity, helping it associate existing DOM nodes with the same product after new results arrive. A database ID is more suitable than a changing position for these lists. @empty supplies the no-results view.

## Aliases make nullable values easier to use

@if (order(); as order) renders the block only for a truthy order and gives that block a local order variable. @if (nextStatus(); as next) does the same for the next allowed status. These names exist in the template block, not as additional class properties.

## Structural directives also appear

The built-in @if and @for blocks do not need CommonModule imports. The project's *appPermission directive does need to be imported into the component. Its leading * expands into template-based conditional rendering; it is a custom construct, not another built-in @ block.

15 / FOUNDATIONS

# Pipes and formatting

CODE MAP: products.ts; orders.ts; products.component.html; orders.component.html

A pipe transforms a value for display without changing the stored product or order. The pipe character is Angular template syntax, separate from an Observable's .pipe(...) method.

Project examples:

```
{{ product.price | currency: "INR" }}
{{ order.createdAtUtc | date: "medium" }}
```

CurrencyPipe formats a number using a currency code and locale rules. INR requests Indian rupees, but does not by itself set the entire application to an Indian locale. The reviewed app does not register a custom LOCALE_ID. Separators, rounding display and symbol placement follow the configured/default locale.

DatePipe formats timestamps for presentation. The order list uses the medium format and no explicit timezone argument, so the display uses the environment's local timezone by default. The server field createdAtUtc represents a UTC timestamp; formatting does not change the persisted timestamp.

## Standalone dependencies

Products imports CurrencyPipe in its component metadata. Orders imports both CurrencyPipe and DatePipe. A missing pipe import can cause a template compilation error even if you imported the symbol at the top of the TypeScript file.

## Display does not establish business correctness

Formatting a number to two visible decimals is not a substitute for monetary validation. The backend owns order pricing and totals and uses its own persistence constraints. A cart's estimated total is useful feedback, not a trustworthy bill supplied by the browser.

## Other formatting in the templates

product.name.slice(0, 1) produces decorative initial artwork. Conditional expressions display Yes/No or Lock/Unlock labels. These are expressions, not custom pipes. The project defines no custom pipe.

> Checkpoint: changing currency formatting should not alter the price used by the server when an order is created.

16 / FOUNDATIONS

# Reactive forms: individual controls

CODE MAP: products.ts; orders.ts; create.ts; admin/users.ts

A FormControl represents one editable value, its validation status and changes over time. A native input is the visible editor; the FormControl is the model in TypeScript.

Project excerpt:

```
readonly search = new FormControl("", { nonNullable: true });
readonly sort = new FormControl("name", { nonNullable: true });
```

```
<input type="search" [formControl]="search" maxlength="100" />
<select [formControl]="sort">
  <option value="name">Name A-Z</option>
</select>
```

ReactiveFormsModule provides these template directives. search.value reads the current value; search.valueChanges is an Observable of future changes. Calling search.setValue("desk") also updates the connected input and normally emits a change.

## Why nonNullable?

It keeps a control typed as a string rather than string | null. reset() without an explicit value returns to the control's initial value. This lets the API methods accept search.value without repeatedly handling null.

## Search sequence

The component subscribes to valueChanges. debounceTime(300) waits for a 300 ms pause; distinctUntilChanged() skips consecutive equal emitted strings. Its callback resets or defaults pagination to page 1 and calls a normal load method.

The simplified screens load initial data explicitly in ngOnInit(). They no longer rely on startWith or combineLatest to trigger the first search. Sort and status changes load immediately through their own subscriptions.

## User experience detail

While typing, search.value already contains the newest input even before the delayed search callback fires. Any explicit load invoked during that period reads the current control value. Learn to distinguish the current value from the timing of a debounced event.

17 / FOUNDATIONS

# FormGroup, validation and submission

CODE MAP: account.ts; account.component.html; admin/products.ts

A FormGroup groups named controls into one form. Account has email, password and currentPassword. ProductAdmin has name, sku, description, price and stockQuantity.

Teaching example:

```
readonly form = new FormGroup({
  name: new FormControl("", {
    nonNullable: true,
    validators: [Validators.required, Validators.minLength(2)],
  }),
});
```

```
<form [formGroup]="form" (ngSubmit)="save()">
  <input formControlName="name" />
  <button>Save</button>
</form>
```

The formControlName matches a key in the group. ngSubmit supports normal form submission, including the keyboard. A secondary button inside a form needs type="button" when it should not submit.

## State and helper methods

- invalid checks whether validation fails. touched records interaction; markAllAsTouched() makes validation feedback visible after submission.
- getRawValue() returns all controls, including disabled ones. A group's ordinary value can omit disabled controls.
- patchValue(product) copies matching properties into the form. setValue expects the complete control shape.
- reset() clears interaction state and restores defaults; ProductAdmin supplies explicit reset values.

## Validation actually used

Required, email, minimum/maximum length, numeric range and regular-expression pattern validators appear in the project. strongPassword returns null when valid or { passwordStrength: true } otherwise. New passwords require length 12 plus lowercase, uppercase, number and symbol. Login's client validator only requires a password; the backend still applies its own request constraints.

busy prevents duplicate UI submissions. Browser validation helps users; server validation remains mandatory for every request.

18 / FOUNDATIONS

# HTTP services and typed API contracts

CODE MAP: product-api.ts; order-api.ts; core/auth.ts

HttpClient makes asynchronous HTTP requests and returns Observables. ProductApi and OrderApi give those requests application-specific names.

Project excerpt:

```
list(search: string, sort: string, pageNumber: number) {
  return this.http.get<Page<Product>>("/api/products", {
    params: { search, sort, pageNumber, pageSize: 12 },
  });
}
```

The query parameters become part of the URL. HttpClient encodes them, so callers should not manually concatenate unescaped search text. The relative /api URL works with the development proxy and with same-origin production hosting.

| HTTP method | Project use |
| --- | --- |
| GET | List products/orders/users; read current user and order |
| POST | Login, register, create order, add item, cancel order |
| PUT | Update product/item/status/user role or lock |
| DELETE | Deactivate product or remove an order item |

## Request versus response types

Product contains id, name, sku, description, price, stockQuantity, isActive and rowVersion. Order contains customerId, timestamps, status, totalAmount, rowVersion and item DTOs. Creating an order sends productId and quantity, not the entire Product object or a trusted total.

## Version strings

rowVersion is an opaque server-generated concurrency value represented as a string. Send back the version you loaded when editing. HttpClient's params handling safely encodes versions containing characters such as + or /.

## Success bodies differ

Order mutations return an updated Order DTO, so OrderDetail replaces its stored order. Product saves can return no body, so ProductAdmin reloads the list. Do not assume every successful request has a JSON payload.

19 / FOUNDATIONS

# Observable, subscribe and Subscription

CODE MAP: products.ts; order-api.ts; core/auth.ts

An Observable describes a stream of values over time. A subscription listens to it. HttpClient Observables normally emit one response and complete; form valueChanges can emit repeatedly.

Project-style example:

```
this.productRequest = this.api.list("desk", "name", 1).subscribe({
  next: result => {
    this.data.set(result);
    this.loading.set(false);
  },
  error: () => {
    this.error.set(true);
    this.loading.set(false);
  },
});
```

next receives a value. error receives a failure and ends that subscription. complete is an optional callback for normal completion; it is not automatically called after an error.

## Cold HTTP requests

Calling api.list(...) builds an Observable. Subscribing sends the HTTP request. Two independent subscriptions to a normal HttpClient Observable normally produce two requests. Merely assigning it to a variable does not share the result.

## Asynchronous order

A line after subscribe() normally runs before the server response arrives. Do not expect this.data() to contain the new response immediately after starting a request. Put response-dependent work in next, or use the signals after they are updated.

## Subscription cleanup

subscribe() returns a Subscription. unsubscribe() stops listening and runs teardown. The simplified catalog stores its active request so the next load can cancel it. Subscription.add groups the search/sort listeners for cleanup.

## Promise comparison

A Promise resolves or rejects once. An Observable can emit multiple values, supports operators and has an unsubscribe mechanism. firstValueFrom bridges an Observable to a Promise for the app initializer. The project does not convert every API request to async/await.

20 / FOUNDATIONS

# RxJS operators: the small set to learn first

CODE MAP: products.ts; orders.ts; admin/products.ts; core/auth.ts

Observable.pipe(...) applies operators to a stream. It is unrelated to the currency/date pipe in a template.

| Operator/helper | Plain meaning | Project example |
|---|---|---|
| debounceTime(300) | Wait for a pause before emitting | Search text |
| distinctUntilChanged() | Skip repeated consecutive values | Search text |
| tap(callback) | Perform a side effect; keep the value | Store current user |
| map(callback) | Transform the emitted value | Preserve the login user result |
| catchError(callback) | Replace or rethrow a failed stream | Restore anonymous startup |
| of(null) | Emit null, then complete | Safe unauthenticated fallback |
| throwError(factory) | Produce an error Observable | Propagate HTTP failure |
| finalize(callback) | Run cleanup on completion/error/unsubscribe | Decrement pending requests |
| takeUntilDestroyed(ref) | Stop at component destruction | Search and API cleanup |

## Array map is not Observable map

cart().map(...) transforms each array element immediately. An RxJS map transforms each value emitted by a stream. The name is similar, but the inputs and timing differ.

## Why error placement matters

If an outer form event stream fails permanently, future typing would no longer produce requests. The simplified components avoid this problem by starting an independent HTTP subscription in load(). A failed request updates error state while the form listener stays alive.

Auth.restore() instead uses catchError to replace startup failure with null. The interceptor uses catchError to retry eligible requests and then rethrows a final failure so the screen can show its own message.

> Learn these operators by following one existing use of each. Do not rebuild a complex combined stream before the direct methods make sense.

21 / FOUNDATIONS

# Advanced RxJS retained for authentication

CODE MAP: core/auth.ts; core/http.ts; app.config.ts

Most screens now use direct methods. Authentication still coordinates several requests whose order and sharing matter.

## switchMap

switchMap takes an emitted value and subscribes to the next Observable. It replaces the previous inner subscription if the outer stream emits again. In the one-response HTTP chains here, it runs the next dependent HTTP step after the previous step succeeds.

Auth.refreshSession() performs:

```
GET csrf
  -> POST refresh
  -> GET csrf for the updated identity
  -> GET me
  -> advance the session version
```

## shareReplay

refresh() saves the in-flight Observable in refreshing. shareReplay({ bufferSize: 1, refCount: false }) lets callers wait on one refresh and receive its result. Keeping refCount false means one caller leaving does not cancel the shared refresh for other waiting requests.

finalize is before shareReplay so clearing refreshing is tied to the shared source finishing, rather than to each individual subscriber leaving. Moving that cleanup can break request coordination even if a single manual login still works.

## firstValueFrom

The initializer needs a Promise Angular can await. firstValueFrom subscribes, resolves with the first value and unsubscribes. restore() returns a User or null on the handled paths, allowing startup to finish.

## Older concepts you may see in notes

Subject emits explicit events. combineLatest combines the latest values from several sources after each has emitted. startWith supplies an initial emission. Earlier versions of the list screens used these; the current simplified list implementations do not. They are not prerequisites for reading Products now.

> Advanced checkpoint: sharing an Observable is a deliberate behavior. Storing a normal cold Observable in a variable alone does not merge requests.

22 / FOUNDATIONS

# Routing and lazy-loaded pages

CODE MAP: app.config.ts; account.ts; detail.ts; app.component.html

The router maps the browser URL to the component displayed in router-outlet. Navigating with routerLink usually changes the view without requesting an entirely new HTML document.

Project excerpt:

```
{
  path: "orders/:id",
  canActivate: [signedInGuard],
  loadComponent: () =>
    import("./features/orders/detail").then(m => m.OrderDetail),
}
```

loadComponent uses dynamic import to load page code when needed. The Promise selects the exported OrderDetail class. This can keep feature code out of the initial bundle.

## Three URL data sources

Path parameters identify a resource: /orders/42 gives id = "42". Query parameters add navigation options: /account/login?returnUrl=/orders. Fragments follow #: recovery links carry email/token there rather than in the HTTP request path.

OrderDetail reads route.snapshot.paramMap.get("id") and converts it to a number. Account reads snapshot.data["mode"], queryParamMap and fragment. A snapshot is a one-time read. If a future design reuses the same component instance while parameters change, subscribe to route changes instead of assuming ngOnInit runs again.

## Navigation methods

routerLink="/products" is a static link. [routerLink]="['/orders', order.id]" builds a dynamic link. Router.navigate takes route commands; navigateByUrl takes a URL string. These methods return Promises.

## Order matters

The empty path uses pathMatch: "full" for the redirect. orders/new precedes orders/:id so "new" is not interpreted as an ID. The ** catch-all belongs last. Access-denied and not-found are small route components with separate templates.

23 / FOUNDATIONS

# Guards and safe navigation

CODE MAP: core/guards.ts; account.ts; app.config.ts

A CanActivateFn answers whether a route can activate. The project's guards return true or a UrlTree representing a redirect.

Teaching excerpt based on signedInGuard:

```
const auth = inject(Auth);
const router = inject(Router);
if (auth.signedIn()) {
  return true;
}
return router.createUrlTree(["/account/login"], {
  queryParams: { returnUrl: state.url },
});
```

Returning the redirect lets Angular manage navigation. Do not start a separate navigation and then return true, which would permit the original route at the same time.

## Permission guard

For data: { admin: true }, permissionGuard calls auth.isAdmin(). Otherwise it checks the permission named in route data. Failure returns /access-denied. Route data is static metadata supplied by app.config.ts, not a permission grant from the server.

## Return after login

Account.goToReturnUrl() reads returnUrl and accepts a value beginning with / but not //. Otherwise it navigates to /products. This keeps the application's post-login redirect within its intended route path rather than blindly trusting a supplied external URL.

## Restoration before navigation

An initially empty user signal does not prove the browser has no valid session. The app initializer waits for Auth.restore(), so the first protected route is not rejected merely because memory was recreated during reload.

## Separate HTTP and navigation failures

A guard denial changes navigation. A server 403 is a failed HTTP request and is handled by the interceptor/component. Both may represent insufficient access, but they run at different layers. Editing route code in the browser cannot grant API permissions.

24 / FOUNDATIONS

# Shared components, inputs, outputs and directives

CODE MAP: shared/ui.ts; feedback.component.html; pager.component.html

Feedback renders error and success messages. Pager renders a page label and Previous/Next buttons. Permission controls whether its enclosed template is rendered.

## Inputs: values from parent to child

Pager declares page = input(1) and pages = input(1). Feedback declares error = input("") and success = input(""). These are input signals read with parentheses inside the child template.

Project-style parent template:

```
<app-pager
  [page]="data().pageNumber"
  [pages]="pages()"
  (change)="load($event)"
/>
```

## Outputs: events from child to parent

Pager declares change = output<number>(). Its button calls change.emit(page() + 1). The parent's $event is that page number, not a DOM click object. The child requests a change; the parent owns the query and performs the HTTP request.

## Permission as a structural directive

*appPermission="'ManageProducts'" passes the string ManageProducts into an input.required<string>(). The outer quotes delimit the HTML attribute; the inner quotes make the Angular expression a string literal.

TemplateRef represents the enclosed template. ViewContainerRef is the location where an embedded view is inserted. An effect clears that location, checks permission and creates the view when allowed. Admin is a special role check; other values check permission names.

Feedback is presentation-only and does not perform HTTP requests. Pager does not fetch data. Permission does not enforce backend security. Each shared building block has one small responsibility.

> Checkpoint: explain why changing a child's page input directly is not how Pager should tell the parent to fetch another page.

25 / FEATURE WALKTHROUGHS

# The root component and shared layout

CODE MAP: app.ts; app.component.html

App is the root component attached to app-root. It imports RouterLink, RouterOutlet and Permission for its template. The injected Auth and HttpState services are public because the HTML reads them; Router and DestroyRef are implementation details.

## What each layout area does

- Header brand and Catalog link navigate to /products.
- auth.signedIn() chooses the signed-in or signed-out navigation.
- Permission directives filter Products admin and Users links.
- auth.user()?.email safely displays the current user's email.
- http.pending() shows the top progress bar for tracked API activity.
- error() shows a root logout failure message.
- router-outlet hosts the selected page; footer displays static stack information.

## Sign out

logout() clears the local error, subscribes to auth.logout(), and navigates to /account/login after success. A failure stores a readable message. takeUntilDestroyed handles cleanup if the component is destroyed.

Auth.logout() obtains CSRF, posts logout, clears user state and obtains a new CSRF token for the now-anonymous identity. The root component does not delete cookies manually or read authentication tokens.

## Two error states to distinguish

HttpState.error stores a general non-auth HTTP error in the interceptor. App.error stores the root component's own message. The reviewed root template reads App.error, not HttpState.error. Feature screens independently display their own local errors through Feedback or inline markup.

## Why the layout stays visible

Route changes replace content at router-outlet rather than reconstructing the whole document. A missing/commented outlet prevents page content appearing even if navigation and API setup are otherwise correct. The current reviewed outlet is active.

26 / FEATURE WALKTHROUGHS

# Products.ts: the first complete class to learn

CODE MAP: features/products/products.ts; product-api.ts

Products is your simplest full example of a searchable, paged HTTP screen.

| Member | Responsibility |
| --- | --- |
| search, sort | Hold user filter values |
| data | Hold the current Page<Product> response |
| loading, error | Control visible request states |
| page | Hold the selected page number |
| pages | Derive total page count from data |
| ngOnInit() | Load initial products and register listeners |
| applyFilters() | Reset page to 1 and load |
| loadProducts() | Cancel old read, request data, handle result |
| move(delta) | Validate and select the next/previous page |
| ngOnDestroy() | Unsubscribe from listeners and active request |

## Follow loadProducts in order

First unsubscribe from the previous productRequest. Then set loading true and error false. Read search.value, sort.value and page(), and pass them to ProductApi.list(). On next, replace data and stop loading. On error, set error true and stop loading.

The screen retains previously stored data after failure, but its template shows the error branch until a later load succeeds. Retry simply calls loadProducts() again with the current values.

## Follow move

nextPage = page() + delta. If it falls outside 1..pages(), return immediately. Otherwise set page and reload. The HTML also disables unavailable buttons; the method's bounds check protects its own operation.

## No hidden combined stream

Initial loading, filter changes and pagination all meet at the same loadProducts method. Search uses only debounceTime and distinctUntilChanged before calling applyFilters. This is the pattern to recognize in the other simplified screens.

> Breakpoints: ngOnInit, applyFilters, loadProducts, the next callback and move. Inspect values, not just the line highlighted by the debugger.

27 / FEATURE WALKTHROUGHS

# Catalog HTML and one search traced end to end

CODE MAP: products.component.html; products.ts; product-api.ts

## The template in display order

The introduction explains the catalog. The heading reads data().totalCount. Search and sort bind to FormControls. The loading/error/success branches determine which content appears. @for renders product cards, @empty handles zero matches, and pagination calls move(-1) or move(1).

Each card reads SKU, name, description, price and stockQuantity. The first name character is decorative artwork. currency: "INR" formats price; [class.sold-out] applies a style when stock is zero. Create an order navigates to /orders/new, whose guard requires sign-in.

## Trace a search for "desk"

1. The user types into the input linked to search.
2. valueChanges emits while typing; debounceTime waits for a pause.
3. The subscription calls applyFilters(), selecting page 1.
4. loadProducts() cancels the old read and sets loading/error state.
5. ProductApi requests /api/products with search, sort, pageNumber and pageSize.
6. The interceptor prepares the API request and counts it as pending.
7. The server filters/sorts/pages products and returns JSON.
8. The subscriber stores the page with data.set(result).
9. loading becomes false, so the product grid or empty message renders.

## What to inspect in DevTools

Find the request's query parameters and status. Inspect response.items and totalCount. Confirm that switching sort requests page 1. A search pause can still lead to separate requests if you type again after the delay; debounce is not a promise of exactly one request for an entire editing session.

> Practice: force an API failure, observe the retry UI, restore the API and click Try again. The page should recover without being reloaded in the browser.

28 / FEATURE WALKTHROUGHS

# Account component: one form, several operations

CODE MAP: features/account/account.ts; account.component.html

The router reuses Account for seven account routes. Each route supplies data.mode. Account reads it to choose the title, fields, validators, request and success behavior.

## State and setup

mode comes from route.snapshot.data. accountTitles maps it to a heading. needsPassword controls whether a password field is required. fragment is parsed with URLSearchParams; token and email can come from an account recovery link. The form contains email, password and currentPassword even when some controls are not displayed.

ngOnInit removes a token-bearing fragment from browser history with replaceUrl after capturing it in memory. Hidden fields have mode-dependent validators so they do not incorrectly block forms that do not need them.

## Method map

- passwordValidators(): no password for email-only operations; required only for login; required plus strongPassword for new passwords.
- submit(): touch fields, reject invalid/busy submissions, clear feedback and subscribe once.
- createRequest(): select login, change-password or a generic auth endpoint/body.
- handleSuccess(): select navigation, password-updated feedback or email instructions.
- goToReturnUrl(): accept the allowed local return path or use /products.

## Template connection

[formGroup] connects the group; formControlName selects controls. @if shows the relevant email/current/new password fields. autocomplete distinguishes email, current-password and new-password. [disabled]="busy()" prevents repeated clicks during the request.

## Validation feedback

The template shows a general validation message when the form is touched and invalid. It does not currently show a distinct message for every individual control error. Client checks and strongPassword improve feedback; Identity and backend DTO validation remain authoritative.

29 / FEATURE WALKTHROUGHS

# Account operations and their outcomes

CODE MAP: account.ts; core/auth.ts; API Controllers/AuthController.cs

| Mode | Main input | Successful UI outcome |
|---|---|---|
| login | email, password | Navigate to a safe return URL or catalog |
| register | email, password | Generic instruction to check email |
| forgot-password | email | Generic instruction to check email |
| reset-password | email, token, new password | Clear local user, reset form, sign-in message |
| change-password | currentPassword, newPassword | Clear local user, reset form, sign-in message |
| confirm-email | email, token | Email-confirmed message |
| resend-confirmation | email | Generic instruction to check email |

## Registration and confirmation

Public registration creates a Customer account on the backend; the form does not select an elevated role. Email confirmation is required before login. In development the project supplies a private mailbox mechanism; real delivery is a separate server concern.

## Recovery links

Email/token are carried in a URL fragment. Fragments are not sent as part of the HTTP request path. Angular reads them, removes the visible fragment using replacement navigation, and supplies the token in the appropriate submitted body. Treat recovery tokens as sensitive even though the guide does not show real values.

## Generic responses

Forgot-password, resend-confirmation and registration acknowledgements avoid revealing whether a particular email has an eligible account. A generic success message does not prove a new account was created or mail was sent for every supplied address.

## Failure and retry

The error callback resets busy and displays error.error?.title or a fallback. Keeping the request construction and success handling in separate methods makes it easier to test each mode without reading nested conditional expressions.

> Checkpoint: the frontend login validator intentionally differs from the new-password validator. The API can still reject a password that violates its own DTO constraints.

30 / FEATURE WALKTHROUGHS

# Auth service: user state, login and restoration

CODE MAP: core/auth.ts; app.config.ts; API Controllers/AuthController.cs

Auth is shared across the app. user is User | null; signedIn is computed from whether a user exists. has(permission) checks the user's permission array; isAdmin() checks roles. clear() removes only the in-memory user state.

## Login request sequence

1. GET /api/auth/csrf obtains a request token.

2. POST /api/auth/login submits email/password with CSRF protection.

3. tap stores the returned User and increments epoch.

4. GET csrf obtains a token for the new authenticated identity.

5. map preserves the original User as the login flow's result.

The server writes access/refresh cookies. Angular stores the user description, not those authentication token values. HttpOnly cookies cannot be read by this JavaScript code.

## Restoration after reload

restore() calls me(), which requests /api/auth/me and stores the returned user. If me returns 401, it attempts refresh(). A failed refresh or another handled startup error resolves to null instead of preventing the public app from loading.

The initializer waits for the first restore result. That avoids treating every reload of a protected URL as a new anonymous visit while restoration is still in progress.

## Generic account requests and logout

submit(endpoint, input) obtains CSRF and posts to /api/auth/{endpoint}. logout() uses submit("logout", {}), clears user state after success and obtains a new CSRF token. Clearing the signal alone is not equivalent to revoking a server session or deleting cookies.

## epoch is a session version

It counts successful establishment/refresh of authentication in this service. It is not the JWT itself, not a timestamp and not the database rowVersion used by orders.

31 / FEATURE WALKTHROUGHS

# HTTP interceptor, loading count and error propagation

CODE MAP: core/http.ts; app.config.ts; app.component.html

An interceptor surrounds HttpClient requests with shared behavior. app.config.ts registers apiInterceptor using withInterceptors.

## Trust and preparation

trustedApi accepts the intended relative /api/ paths and rejects other URLs. Untrusted requests continue without this custom behavior. prepareApiRequest clones the immutable request with withCredentials: true. For methods other than GET/HEAD/OPTIONS, it reads the current XSRF token and adds X-XSRF-TOKEN when a token is available.

Reading the token again on retry matters because refresh can change the authenticated identity. Configured built-in XSRF support uses the same cookie/header names; the custom preparation also ensures the retry uses the current request token.

## Request lifecycle

The interceptor increments HttpState.pending when a tracked request starts. finalize decrements it on success, error or cancellation. A counter allows overlapping requests; a boolean could incorrectly hide loading when only one of several requests finishes.

## Eligible 401 handling

It does not refresh for non-401 errors, SKIP_REFRESH requests or /api/auth/ endpoints. If another refresh already advanced epoch, it retries using current cookies. Otherwise it waits on auth.refresh() and retries once through next(). That downstream call does not re-enter this interceptor's retry handler.

## Final errors

A later catchError stores a general non-auth error title and rethrows it. The feature subscriber can then show its local error. Auth endpoint failures are left to the account flow. A 403 means insufficient authorization and is not repaired by repeating refresh.

> Do not remove finalize when simplifying HTTP code: cancelled searches must still decrement the global pending count.

32 / FEATURE WALKTHROUGHS

# Cookies, CSRF and shared refresh

CODE MAP: core/auth.ts; core/http.ts; API Security/AuthSupport.cs

## The browser security pieces

The server's access cookie contains a signed JWT and the refresh cookie contains an opaque refresh token. Both use HttpOnly and Secure. The readable XSRF-TOKEN cookie carries a CSRF request token; Angular copies it into the agreed header for writes. HttpOnly, HTTPS and CSRF address different concerns.

The backend also keeps its internal antiforgery cookie. The readable request token is not interchangeable with an access/refresh token. CORS is not a substitute for CSRF validation. Cookies are scoped by host/path rather than port; switching localhost run modes can create confusing cookie overlap.

## Why one shared refresh is necessary

Imagine Orders and an admin request both receive 401. If each independently rotates the same single-use refresh token, they can conflict. Auth.refresh() saves a shared in-flight Observable, so both wait for one rotation within the same tab.

```
Request A gets 401 ----+
                       +--> one refresh sequence --> retry A and B
Request B gets 401 ----+
```

After successful refresh, epoch advances. A late 401 from a request sent before that refresh can retry with the new cookies without rotating again. A failed refresh clears user state and propagates the error. finalize releases the saved handle for a future refresh attempt.

## Limits to understand

This service coordinates within one tab, not between independent tabs. The server retains consumed-token evidence for reuse detection. Revoking refresh sessions does not instantly rewrite all already-issued access claims; those can remain usable until expiry. The configured access lifetime defaults to five minutes.

> Advanced checkpoint: preserve the refresh ordering, sharing and error propagation even when the surrounding code is made easier to read.

33 / FEATURE WALKTHROUGHS

# Orders list: filters and pagination

CODE MAP: features/orders/orders.ts; orders.component.html; order-api.ts

Orders follows the simplified catalog idea with three controls: search, status and sort. The default sort is newest; the statuses list contains Draft, Confirmed, Shipped, Completed and Cancelled.

## Initialization

ngOnInit loads orders immediately. It subscribes to debounced search and immediate status/sort changes. Each filter listener calls loadOrders() without a page argument, so the method defaults to page 1.

## loadOrders(page = 1)

It cancels orderRequest, sets loading true, clears the error and calls OrderApi.list with page and current controls. On success it stores the page and stops loading. On error it stores the title/fallback and stops loading. The form listeners survive a request failure.

## Template behavior

The heading checks ViewAllOrders to display All orders or Your orders. Fields bind to the controls. Rows display ID, formatted timestamp, status and INR total. The Details link builds /orders/{id}. @empty supplies a no-results table row.

The reusable Pager receives data().pageNumber and pages(), and emits a page number to loadOrders($event). The server response determines the displayed page. Unlike Products, this screen does not store an additional page signal.

## Error detail

The current template displays Feedback above the table. After an error, previously stored table data can remain visible because the template hides it only while loading, not whenever error is nonempty. Learn this behavior rather than assuming every screen uses the catalog's three exclusive branches.

## Permissions and ownership

The frontend label is convenience. The backend decides whether the caller receives only their own orders or all orders from the ViewAllOrders claim. Supplying another user's ID in a client query is not how access is granted.

34 / FEATURE WALKTHROUGHS

# CreateOrder: catalog selection and local cart

CODE MAP: features/orders/create.ts; create-order.component.html

CartLine contains a Product and a quantity. The cart signal is a local preview before an order exists on the server. Reloading or leaving this component does not persist that unfinished cart.

## Loading selectable products

ngOnInit calls loadPage(1) and registers a debounced search. loadPage cancels the old product request, sets productPage, clears error and loads a catalog page. Success updates products and productPages. The shared Pager emits the next page into this method.

## Cart methods

- add(product): find an existing line; increase its quantity if found, otherwise add a new line with quantity 1.
- quantity(id, quantity, input?): require an integer from 1 to 10000; restore the input display on invalid input; create updated lines on success.
- remove(id): keep only lines with other product IDs.
- total: compute the estimated sum of price multiplied by quantity.

The template disables Add for out-of-stock products, reads quantity from #qty and displays line previews. Those UI checks do not reserve stock or promise availability at submission time.

## Creating the Draft

create() rejects a duplicate busy action, clears error and maps the cart to { productId, quantity } objects. OrderApi.create sends { items }. Success navigates to the returned order ID. Failure clears busy and displays a server title or fallback.

Only IDs and quantities are sent as the order's input. The server validates active products, available stock, line limits and prices. Stock is reserved when the Draft is created, not merely when a local cart line is added.

> Trace: add two products, change one quantity, inspect the outgoing JSON and compare the returned server total with the estimated preview.

35 / FEATURE WALKTHROUGHS

# OrderDetail: edits, cancellation and status

CODE MAP: features/orders/detail.ts; order-detail.component.html; order-api.ts

OrderDetail loads the order identified by the route and loads selectable products. order is nullable until a response arrives. @if (order(); as order) provides the template's non-null local value.

## Derived action availability

canEdit returns true only for a Draft owned by the current user or accessible for editing as Admin. nextStatus maps Draft -> Confirmed, Confirmed -> Shipped, and Shipped -> Completed. Permission markup separately limits the staff status action. Backend rules run again for every mutation.

## Explicit action methods

add and update first validate integer quantity 1..10000 and check that an order exists. remove, cancel and changeStatus each obtain the current order and build the corresponding API request. The template calls these named methods rather than constructing HTTP calls itself.

## The common run(request) helper

It rejects duplicate busy operations, clears feedback and subscribes once. On success it replaces order with the returned DTO, clears busy and shows Order updated. On failure it clears busy and shows a title or reload-before-retrying message.

Replacing the response is important: the next request must use the latest parent rowVersion. Retaining the old DTO can make a later valid edit fail with a stale-version conflict.

## UI rules you can observe

Draft item quantities can be edited. The last remaining line cannot be removed using the UI. Cancellation appears for Draft/Confirmed. Status advancement appears only with ManageOrderStatus and a next status. Reload re-reads the order. Find explicitly searches the first catalog page; it is not a debounced search control here.

> A read request may be cancelled safely by a newer read. Cancelling a write subscription does not guarantee the server rolled the change back.

36 / FEATURE WALKTHROUGHS

# Product administration: CRUD with a soft delete

CODE MAP: features/admin/products.ts; product-admin.component.html

ProductAdmin uses the protected /api/products/admin endpoint, which can include inactive products. Its search listener calls load(), and Pager calls load(page). Each read cancels the previous list request.

## Editing state and forms

editing stores the original Product DTO, including its version. edit(product) selects that DTO and patchValue fills matching form controls. reset() clears editing and resets explicit defaults for a new product.

The form validates name length, uppercase SKU pattern, description length, positive bounded price and bounded nonnegative integer stock. The backend imposes additional constraints such as SKU uniqueness and price precision. Angular's stock validator includes an integer pattern.

## Save sequence

1. Mark all fields touched and reject invalid/busy submission.
2. Set busy and clear the error.
3. Read the form and attach the originally loaded rowVersion.
4. Choose PUT for an existing ID or POST for a new product.
5. On success clear busy, show feedback, reset the editor and reload.
6. On failure clear busy and display the server title/fallback.

## Deactivation

deactivate(product) sends DELETE with rowVersion as an encoded query parameter. The operation is a soft deactivation: IsActive changes while historical order references remain. It is not a physical removal of every record related to the product.

## Concurrency exercise

Open a product for editing in two sessions. Save in one, then submit the stale editor in the other. A 409 means the stored version changed. Reload and review before retrying; do not simply replace rowVersion while silently keeping stale values.

> Important distinction: the public catalog excludes inactive products; the protected admin list is allowed to show them.

37 / FEATURE WALKTHROUGHS

# User administration: roles and locks

CODE MAP: features/admin/users.ts; users.component.html; API AdminController.cs

Users loads available role names from /api/admin/roles and a paged user list from /api/admin/users. The role choices come from the backend. Search is debounced; load(page) cancels older reads and uses the current email search text.

AdminUser includes id, email, emailConfirmed, locked and roles. Its TypeScript interface describes the expected list response, not a database entity exposed for arbitrary editing.

## Changing a role

The template's #role select provides role.value. assign(user, role) calls update(user, "role", { role }). The method sends PUT /api/admin/users/{id}/role and reloads the current page after success.

## Locking an account

lock(user) calls update with { locked: !user.locked }. The same method handles busy state and feedback, keeping the two actions easy to compare. The label changes between Lock and Unlock according to the stored locked value.

## Self-management and server authority

The template disables role and lock controls when user.id equals auth.user()?.id. The AdminController also rejects self-demotion/self-locking operations. Another authorized administrator is required for those changes; enabling a disabled HTML element does not bypass the API rule.

## Why changes may not appear everywhere immediately

Role changes and account locks revoke refresh sessions. Previously issued access claims can remain until their short expiry. The current UI's Auth.user is a snapshot too, so changing another user's role does not push new state into every open tab.

## Two initial requests

Roles and users are separate reads with separate possible failures. A failed roles lookup is different from a failed user-page request. Use Network to identify which operation failed rather than assuming the whole page has one endpoint.

38 / INTEGRATION AND PRACTICE

# Backend concepts Angular needs to understand

CODE MAP: Application/Contracts.cs; Application/Products.cs; Controllers; Infrastructure

You do not need to master every .NET file before learning Angular. You do need the contract between the browser and API.

## Request binding and DTO validation

ASP.NET controllers bind query strings and JSON bodies into request DTOs. Validation can reject missing or out-of-range values before the operation proceeds. Frontend interfaces should match response field names and value shapes, including string order statuses and numeric IDs.

## Persistence layers

Domain contains Product, Order and OrderItem. Application describes operations/contracts and business services. Infrastructure uses EF Core and SQL Server. Controllers expose those operations as HTTP endpoints and apply authorization.

## Read queries

Repositories apply search filters, stable sorting and pagination. totalCount counts matching rows across pages; items contains only the selected page. Public product reads exclude inactive products. Read-only projections and no-tracking queries reduce unnecessary persistence work.

## Caching

The backend catalog cache is bounded and versioned; it is enabled by default for Development and disabled by default for Production. A repeated request can be served without reaching the repository. This is distinct from an Angular signal holding the last response and from browser network caching.

## Transactions and server checks

Order creation coordinates order rows, item rows and stock in one transaction. Authorization policies, ownership checks, inventory checks and concurrency checks still apply if a caller skips the Angular interface entirely.

## Other server systems you will encounter

Identity handles account/password/role functionality. EF migrations evolve the schema. Data Protection supports protected server tokens/keys. Serilog records structured operational logs. These are supporting backend systems, not Angular imports or browser state-management tools.

39 / INTEGRATION AND PRACTICE

# Business rules: totals, stock and order status

CODE MAP: Application/OrderService.cs; Domain/Order.cs; order-api.ts

## Prices and totals

The server snapshots each product's price onto a new order item. Later catalog price changes do not rewrite existing item unit prices. The client computes an estimate for usability; the server calculates the saved total.

## Inventory example

Teaching example with a product priced at INR 100 and stock 10:

- Create a Draft for quantity 2: saved total 200; available stock becomes 8.
- Increase that line to 3: total 300; one more unit is reserved, stock becomes 7.
- Reduce to 1: total 100; two units return, stock becomes 9.
- Cancel: reserved stock returns, stock becomes 10; historical items/total remain.

The service uses the quantity difference when editing. It rejects unavailable products, insufficient stock, invalid quantities, duplicate products and inappropriate edits. An order supports at most 100 distinct items in the server contract.

## Transition graph

```
Draft -> Confirmed -> Shipped -> Completed
  |          |
  +----------+------> Cancelled
```

Only eligible staff can advance fulfillment. Owners can cancel eligible own orders; staff have the corresponding server-side ability. Item editing is limited to Draft. A repeated cancellation does not restore stock again. The final line cannot be removed; cancel instead.

## Why rowVersion appears everywhere

It detects a stale edit against a resource that changed since it was loaded. It is not a date, an ID or authentication state. Orders use the parent order version for item changes and transitions. The server also uses transaction/locking safeguards; Angular alone cannot prevent two clients competing for the last unit of stock.

40 / INTEGRATION AND PRACTICE

# Errors, empty results and safe retries

CODE MAP: core/http.ts; feature error callbacks; API Program.cs; Contracts.cs

| HTTP status | Typical project meaning | Useful next action |
|---|---|---|
| 400 | Input or CSRF validation failed | Inspect request fields and CSRF sequence |
| 401 | No usable authentication | Follow restore/refresh or sign in |
| 403 | Caller lacks permission | Verify account role/permissions |
| 404 | Missing resource or hidden foreign order | Check route/resource/access |
| 409 | Stale version, stock conflict or invalid transition | Reload and review current state |
| 500 / 503 | Server failure or readiness unavailable | Check API logs and dependencies |

## ProblemDetails

The backend often returns a JSON error with a title. Components use error.error?.title ?? a fallback. The outer error is Angular's HTTP error object; its error property holds the response body. A network failure may have no server title at all.

## Empty is a successful result

An empty items array can be a 200 response. That should show No matching products/orders rather than a failure message. loading, error and data have distinct responsibilities even when the UI shows only one branch.

## Retry reads deliberately

The catalog's retry button calls the same read method again. Search listeners also remain active after failure. A 409 edit should trigger review of refreshed data, not an automatic blind overwrite. A network timeout during a write can leave the outcome uncertain because the server may already have committed.

## Layered feedback

The root progress bar counts all tracked API requests, while a screen's loading signal controls its own results. A local busy signal blocks duplicate mutations. These states are related but not interchangeable.

> Checkpoint: why should a 403 not enter the same refresh behavior as an eligible 401?

41 / INTEGRATION AND PRACTICE

# Styling, responsive layout and accessibility

CODE MAP: src/styles.css; all feature templates; index.html

The project uses global CSS in styles.css rather than separate component stylesheets. Classes such as grid, toolbar, table-wrap, notice, primary and pagination provide reusable styling across screens.

## Layout concepts in use

CSS Grid builds product cards. Flexbox aligns navigation/toolbars/actions. Media queries reduce catalog columns from three to two to one on smaller screens and allow navigation/toolbars to wrap. table-wrap adds horizontal overflow so wide tables remain usable. The viewport meta tag lets the page adapt to device width.

## Interactive state

button:disabled changes the visual state as well as the native button behavior. Focus and focus-visible outlines help keyboard users identify the active control. [class.sold-out] adds a class based on stock. The fixed progress bar communicates global network activity.

## Accessibility patterns you can recognize

- Labels wrap inputs so their purpose is associated with them.
- Semantic header, nav, main, section, article and footer organize content.
- role="status" identifies loading or success feedback.
- role="alert" identifies error feedback.
- aria-label names navigation/loading areas.
- aria-hidden hides decorative initial artwork from assistive technology.
- Native buttons and ngSubmit support keyboard interaction.

These patterns are useful foundations, not a claim that the entire app has passed a complete accessibility audit. Test keyboard navigation, zoom, contrast and screen-reader behavior as separate verification work.

## External fonts

styles.css imports Google Fonts. Production font inlining needs network access at build time in this configuration; a blocked download can fail the production build without indicating a TypeScript problem. The PDF uses its own embedded local fonts and can be read offline.

42 / INTEGRATION AND PRACTICE

# Testing: what the project verifies

CODE MAP: core/auth.spec.ts; features/search-flows.spec.ts; tsconfig.spec.json

The Angular unit-test builder uses Vitest and jsdom. jsdom provides a simulated DOM for tests; it is not a complete real browser. The .spec.ts suffix lets the configured test runner discover these files.

## Test tools

TestBed configures Angular dependencies and creates component fixtures. provideHttpClientTesting replaces the real network backend. HttpTestingController finds outgoing requests, lets a test supply a response, and verifies that no unexpected requests remain. fixture.detectChanges() initializes/render-checks a component; fixture.destroy() triggers cleanup.

## Existing authentication tests

The auth suite checks restoration, refresh failure cleanup, a shared refresh for concurrent 401s, late 401 handling, at-most-once retry, no refresh on 403/login failure, credentials/CSRF trust boundaries, guard redirects and password validation.

## Existing search tests

The search-flow suite checks admin pagination cancellation, 300 ms debounce, reset to page 1, recovery after errors and cleanup on destruction. It also verifies current order filters and order-creation catalog cancellation/pagination.

## Fake time

vi.useFakeTimers() controls time in tests. Advancing 299 ms verifies no search request yet; advancing the final millisecond triggers the 300 ms debounce without waiting in real time. Restoring real timers avoids affecting other tests.

## What passing tests do not prove

Mock HTTP tests do not prove SQL transactions, a deployed server, real cookie/SameSite behavior or browser layout. Backend tests use real SQL for business/security boundaries. Real-browser and deployment checks remain separate.

In the preceding code refactor, the development build and 15 frontend tests passed. Tests required a single-threaded runner workaround in this restricted environment because default fork workers timed out; that temporary configuration was removed afterward.

43 / INTEGRATION AND PRACTICE

# Debugging a feature without getting lost

CODE MAP: products.ts; core/http.ts; order-api.ts; docs/code-walkthrough.md

## Use one question per breakpoint

For catalog search, pause at applyFilters(), loadProducts() and the success/error callbacks. Check search.value, sort.value, page(), loading(), error() and the response. Do not start with every backend and authentication breakpoint enabled at once.

## Browser tools

Use Elements to inspect rendered markup/classes; Console for thrown errors; Network for URLs, query parameters, request bodies, statuses and response shapes. During authentication, inspect flags and request sequence without copying real tokens, passwords or cookie values into notes.

| Symptom | First place to inspect |
| --- | --- |
| Header renders but page is blank | router-outlet and route match |
| Unknown formControl/pipe binding | Component imports array |
| Request never sends | subscribe() and request-building code |
| Search sends too often | valueChanges listener and debounce |
| Old data replaces new data | Active read cancellation |
| Loading never stops | Success/error paths and finalize |
| Repeated 401 requests | Refresh exclusions, sharing and retry rules |
| Edit returns 409 | DTO rowVersion and current server state |
| Production font error | Network access for styles.css font import |

## A reliable investigation loop

Predict one result, perform one action, inspect the request, inspect the callback, then inspect the view. If the request is correct but the view is wrong, focus on signals/template state. If the request is wrong, focus on controls, routing and request construction before investigating SQL.

## Avoid accidental side effects

Search and GET debugging are read-only. Retrying create/order edits can change persistent state. If a write response was lost, inspect current server state before assuming nothing happened and clicking again.

44 / INTEGRATION AND PRACTICE

# Running, building and configuration

CODE MAP: package.json; angular.json; tsconfig.json; proxy.conf.json; README.md

Run frontend commands from D:\Angular ASP\frontend. The npm scripts are defined in package.json.

```
npm ci
npm start
npm run build -- --configuration development
npm test
npm run build
```

npm ci installs the lockfile dependencies; it requires package access and changes node_modules. npm start serves HTTPS on localhost:4200 using the local certificate/key paths in package.json. It assumes the project's development setup and API are ready; it does not initialize SQL or create all certificates itself.

## Two local processes

Source mode uses the Angular development server on HTTPS 4200 and the API on HTTPS 7043. proxy.conf.json sends /api/** to that API. Its secure:false setting is a local development proxy certificate behavior, not a recommendation to disable production TLS checks.

For full first-time backend/database/certificate setup, follow README.md and scripts/Initialize-Development.ps1. Do not copy ignored secret files into this guide or source control.

## Build modes

The development configuration disables optimization and enables source maps. The default production configuration uses hashing and bundle budgets. Output is under frontend/dist/orders; browser assets are used by hosting. Source maps help debugging; a production bundle is generated output, not the TypeScript you edit.

## Strict checks

tsconfig.json enables strict TypeScript and strictTemplates. These catch mismatched types and template mistakes during builds. npm test uses tsconfig.spec.json with Vitest globals. Angular CLI compatibility should be checked against the versions pinned in this repository, not assumed from a newer tutorial.

45 / INTEGRATION AND PRACTICE

# Hosting and delivery: enough context for Angular

CODE MAP: Dockerfile; compose.yaml; azure-pipelines.yml; infra/main.bicep; docs/azure-devops.md

## Development and hosted paths

In source mode, a dev proxy connects the frontend and API. In the Docker image, Angular's built browser files are copied into ASP.NET's wwwroot, and /api is served on the same application origin. Angular still uses relative /api URLs in both modes.

## SPA fallback

Opening /orders/42 directly asks the server for that URL first. Hosting must return the Angular entry document for valid client routes while keeping API failures as API responses. A missing SPA fallback can cause a server 404 before the Angular router gets a chance to run.

## Multi-stage container build

One stage builds Angular with Node; another publishes .NET; the final runtime copies only published outputs and static browser files. The runtime runs as a non-root user. Compose combines the app, SQL, migration job and Seq with health checks and persistent volumes.

## Pipeline concepts

The repository pipeline separates backend tests, frontend tests/build and container smoke checks. Optional Azure release steps use an immutable commit-tagged image and configuration/secrets outside source. Bicep describes infrastructure. A compiled template or successful local build is not evidence that a cloud deployment has already happened.

## Practical limits

The project deliberately targets ASP.NET Core 6 for practice and includes a production-readiness gate for the unsupported framework. This guide does not remove that gate or claim the app is production-ready. Treat upgrading/testing the server runtime as separate work before real deployment.

Google Font download restrictions can fail a production frontend build in this environment. The successful development build checks compilation, but it is not a substitute for a later complete production build and browser smoke test in an appropriately configured environment.

46 / INTEGRATION AND PRACTICE

# A guided learning sequence

CODE MAP: All frontend source; docs/learning-guide.md

Study for understanding rather than finishing a fixed number of pages. Each session ends when you can explain the checkpoint without reading the answer.

| Session | Files and task | Explain before moving on |
|---|---|---|
| 1. Startup | index.html, main.ts, app.config.ts, App | How a URL becomes a page |
| 2. Templates | Catalog HTML and root layout | Text, property and event bindings |
| 3. State | Products signals and computed pages | What changes and what is derived |
| 4. Forms | Catalog controls, account FormGroup | Control value versus valueChanges |
| 5. Requests | ProductApi and loadProducts | Why subscribe and cleanup exist |
| 6. Reuse | Pager, Feedback, Permission | Inputs down and events up |
| 7. Account | Account methods and Auth.login | Validation and the login sequence |
| 8. Orders | Orders, CreateOrder, OrderDetail | Local cart versus saved order |
| 9. Admin | ProductAdmin and Users | Edit state, versions and roles |
| 10. Infrastructure | Interceptor, restore and refresh | 401 retry, CSRF and shared refresh |
| 11. Quality | Tests, CSS, debugging | What is verified and what is not |
| 12. Integration | API contracts and hosting | Which rules belong to the server |

## A session routine

Spend a little time reading, then trace one user action. Change one small thing in a practice branch/copy, run the relevant check, and explain the outcome aloud. Take notes in your own words, with file names and method names rather than copied token values or whole source files.

## Readiness for the code walkthrough

You should be able to trace one full catalog request, describe a form submission, explain who owns state and point out where server authority begins. You do not need to reproduce shareReplay or refresh-family locking from memory to begin learning the simpler screens.

47 / INTEGRATION AND PRACTICE

# Practice exercises

CODE MAP: Products; Account; Pager; Orders; tests

Use a practice branch or copy. Make one small change at a time and restore experiments you do not want to keep.

## Level 1: observe and explain

1. Start on /products. Identify the component, template and API service. State which file defines the route.

2. Type a search slowly, then quickly. Explain why the number of requests can differ.

3. Find one signal, one computed value, one FormControl and one plain field. Explain why each is represented that way.

4. Explain every binding on the Create Draft order button.

## Level 2: small interface changes

5. Add a catalog Clear search button that calls search.setValue(""). Explain when the delayed load occurs and why page resets.

6. Display "Showing N products on this page" using data().items.length, then compare it with totalCount.

7. Add a distinct no-orders explanation to the orders empty state without changing HTTP behavior.

8. Add a per-field required-name message to ProductAdmin using its control state. Keep server validation unchanged.

## Level 3: trace failures and invariants

9. Make the API unavailable briefly. Recover the catalog using retry, then confirm later typing still works.

10. Load two product editors and provoke a stale-version conflict. Explain why automatic overwrite would be wrong.

11. Inspect a create-order request and prove it sends IDs/quantities rather than a trusted total.

12. Read the concurrent-401 test. Predict the requests before reading finishRefresh().

## Extension exercise

Add a new catalog sort only as a full vertical change: select option, allowed API sort and repository ordering. A frontend option alone does not make an unsupported server sort valid. Add a meaningful check, run it and explain the behavior across the boundary.

48 / INTEGRATION AND PRACTICE

# Self-check answers and common misconceptions

CODE MAP: Current simplified frontend implementation

## Answers to the core checkpoints

- The route comes from app.config.ts, the catalog view from products.component.html, behavior from products.ts and HTTP from product-api.ts.
- debounce waits for a pause; typing again after a pause can start another request. It is not a permanent cache or exactly-one-request guarantee.
- Signals hold current UI state. computed derives a value. FormControl represents input and validation. Plain fields fit fixed values that do not need reactive updates.
- [disabled]="!cart().length || busy()" disables an empty or busy submission. (click)="create()" handles the action; the interpolation chooses Creating or Create Draft order.
- items.length counts the current page. totalCount counts all matching items across pages.
- A failed independent HTTP request does not kill the search valueChanges subscription in the simplified implementation.
- rowVersion detects stale resource state. Reload and review instead of silently replacing the version and overwriting another edit.

## Misconceptions to avoid

"readonly means the signal can never change." It prevents reassignment of the property, not .set().

"get<Product>() validates JSON." It provides compile-time expectations; runtime validation is a separate concern.

"A hidden button secures the operation." The API must authorize it.

"A 401 and a 403 are the same." The former may justify session restoration; the latter normally needs different access.

"Unsubscribing a write reverses the database change." The server may already have committed it.

"A currency pipe calculates the saved order total." It formats display; the server owns saved pricing.

"All Angular apps need AppModule and ngModel." This project uses standalone components and reactive forms.

49 / REFERENCE

# Glossary for your walkthrough

CODE MAP: Terms used throughout the frontend

| Term | Meaning in this guide |
|---|---|
| Binding | A connection between template expressions and state/events |
| Bootstrap | Start the root Angular application |
| Component | Class and template forming a view |
| Decorator | Metadata such as @Component or @Injectable |
| Dependency injection | Angular supplies configured dependencies |
| Directive | Behavior attached to an element/template |
| DTO | Data shape exchanged across an API boundary |
| Effect | Reactive side effect that tracks signal reads |
| Guard | Decision before Angular route activation |
| HttpClient | Angular HTTP request service |
| Interceptor | Shared logic around HTTP requests/responses |
| Lazy loading | Load feature code when navigation needs it |
| Observable | Source of values/errors/completion over time |
| Pipe | Template formatting, or an RxJS operator pipeline by context |
| Provider | Configuration telling injection how to supply a dependency |
| Reactive form | Control/group model connected to form inputs |
| Signal | Reactive container with a current readable value |
| Standalone | Component declares its own template dependencies |
| Subscription | Active listener with an unsubscribe mechanism |
| UrlTree | Angular representation of a URL used for navigation |
| CSRF/XSRF | Forged cross-site action risk and its request-token defense |
| JWT | Signed token carrying claims; signing is not encryption |
| Permission | A named capability such as ManageProducts |
| rowVersion | Opaque persisted-resource concurrency version |
| SPA | App whose router updates views within one browser document |

Use the glossary to resume reading, then return to the actual declaration and one concrete use. Knowing the definition is useful; being able to trace the value through the program is the goal.

50 / REFERENCE

# Coverage checklist: concepts and where to find them

CODE MAP: Frontend source inventory reviewed for this document

| Concept family | Actual examples in this project |
|---|---|
| Language | Classes, interfaces, unions, generics, readonly/private, callbacks, destructuring, optional chaining, spread, map/filter/find, Promise |
| Startup | bootstrapApplication, ApplicationConfig, providers, app initializer, firstValueFrom |
| Components | Standalone metadata, selector, templateUrl, imports, root and routed components |
| State | signal, set/update, computed, effect, immutable array/object changes |
| Lifecycle | OnInit, OnDestroy, DestroyRef, takeUntilDestroyed, request cancellation |
| Templates | Interpolation, properties/events/classes, references, $event, @if/@else, @for/@empty, aliases |
| Reuse | input(), input.required(), output(), structural directive, TemplateRef, ViewContainerRef |
| Forms | FormControl, FormGroup, validators, custom validation, touched/invalid, patch/reset/raw values |
| Routing | routerLink, RouterOutlet, dynamic imports, guards, UrlTree, params, queries, fragments, data, fallback |
| HTTP/RxJS | Typed requests, params/body, Observable/Subscription, debounce, distinct, map/tap, error/finalize, switchMap, shareReplay |
| Security | Cookies, XSRF names, HTTP context token, restore, shared refresh, epoch, permissions and server checks |
| Quality | Vitest, TestBed, mock HTTP, fake timers, CSS, semantics/ARIA, strict compilation, proxy/build scripts |

## Deliberately outside this project's implementation

NgRx, Angular Material, server-side rendering/hydration, custom pipes, content projection, ViewChild, component inheritance, dynamic FormArray and template-driven ngModel forms are not used by the reviewed app. Learn them later if a feature calls for them. Older combined-stream code is explained only to distinguish it from the current simplified screens.

This checklist covers the Angular constructs present in the reviewed project. It does not claim to cover every Angular API or every backend/deployment implementation detail.

51 / REFERENCE

# Source index and further reading

CODE MAP: Local source snapshot; official documentation checked 9 November 2026

## Primary source: your project

All frontend paths below are relative to frontend/src/app unless noted otherwise. The explanations of behavior are based on these files, not on an unrelated starter application.

- Startup/layout: frontend/src/index.html, frontend/src/main.ts, app.config.ts, app.ts, app.component.html.
- Infrastructure: core/auth.ts, core/http.ts, core/guards.ts, shared/ui.ts and shared HTML templates.
- Catalog: features/products/products.ts, products.component.html, product-api.ts.
- Account: features/account/account.ts and account.component.html.
- Orders: features/orders/orders.ts, create.ts, detail.ts, order-api.ts and their three HTML templates.
- Administration: features/admin/products.ts, users.ts and their two HTML templates.
- Other routes: access-denied.ts/.component.html and not-found.ts/.component.html.
- Verification/configuration: core/auth.spec.ts, features/search-flows.spec.ts; frontend/package.json, angular.json, tsconfig.json, tsconfig.spec.json and proxy.conf.json; frontend/src/styles.css.
- Backend context: API controllers, Security/AuthSupport.cs, Application/Contracts.cs, Products.cs and OrderService.cs; Dockerfile, compose.yaml, azure-pipelines.yml and existing docs.

## Official references

These are optional follow-up references. The Angular site can evolve beyond the versions pinned here; use versioned Angular 21 material where available and compare examples with the actual project.

- Angular components: https://angular.dev/guide/components
- Signals: https://angular.dev/guide/signals
- Reactive forms: https://angular.dev/guide/forms/reactive-forms
- HTTP interceptors: https://angular.dev/guide/http/interceptors
- Angular 21 lifecycle: https://v21.angular.dev/guide/components/lifecycle
- TypeScript handbook: https://www.typescriptlang.org/docs/handbook/2/everyday-types.html
- RxJS guide: https://rxjs.dev/guide/observable

> Your first walkthrough should begin with the catalog. Bring this guide, pause at each method boundary and explain what you expect the screen to do before running the next step.